

Data visualization tutorial: exploring data and telling stories using ggplot2*

Peter Carbonetto *University of Chicago*

In this lesson, we will learn how to use ggplot2 to create simple yet effective data visualizations. The ggplot2 package is an incredibly powerful plotting interface that extends the base plotting functions in R. [This tutorial is also a Google doc.](#)

Some motivation

A good figure is an important part of an impactful research paper or presentation. A good figure is one that *tells an interesting story*.

Almost inevitably, creating a good figure takes iteration and refinement. You will rarely get it right on the first try.

For these reasons, taking the *programmatic approach* to creating plots is very powerful. It allows you to:

- a. Create an endless variety of plots.
- b. Reuse code to quickly create and revise plots.
- c. Be part of the [Frictionless Research Exchange](#).

In this tutorial we will explore the programmatic approach to plotting using **ggplot2**.

Setup

Do I have what I need? Download the tutorial materials from GitHub, and make sure you know where to find them.

If you have not already done so, install these packages:

```
install.packages("ggplot2")
install.packages("cowplot")
install.packages("ggrepel")
```

If you are running your code in a Jupyter notebook or in Google Colab, I recommend running this line of code so that the outputs look the same as they do in RStudio:

*This document is included as part of the Data Visualization tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version:** August 07, 2025; **Corresponding author:** pcarbo@uchicago.edu. Thanks to Stefano Allesina, John Novembre, Stephanie Palmer and Matthew Stephens for their support and guidance over the years.

```
options(jupyter.rich_display = FALSE)
```

Do smaller dogs live longer?

The study of dogs is a surprisingly fruitful area of research! In this tutorial, we will make use of some data that was made available by the authors of a 2008 *Genetics* article, *Single-nucleotide-polymorphism-based association mapping of dog stereotypes*. These data are stored in a CSV file.

Our main analysis aim is to investigate the anecdotal claim that smaller breeds (such as Chihuahuas) live longer than larger breeds (such as Saint Bernards).

Our first ggplot plot

It will take us some time to understand *how* ggplot works, but let's start by quickly creating our first ggplot.

```
library(ggplot2)
dogs <- read.csv("dogs.csv", stringsAsFactors = FALSE)
ggplot(dogs, aes(x = height, y = weight)) +
  geom_point()
```

A first look at the data

When you load data into R for the first time, it is important to get a basic understanding of the data frame and its contents.

```
# Add your code here.
```

What different types of data are in this table?

The often overlooked scatterplot

In this tutorial, we will learn about ggplot2 through one of the most basic data visualizations: *the scatterplot*.

The scatterplot is easily overlooked because it is so simple. But it can be one of the most effective ways to visualize relationships. And it has many uses.

With embellishments (adding labels, varying color, shape, size, *etc*), scatterplots can produce stunning visualizations.

Our first ggplot2 (with “ugly” code)

Recall, our objective is to investigate the relationship between size and longevity in dogs. We'll use weight as a proxy for size.

```
# Add your code here.
```

Now for some more elegant code that can accomplish the same thing. (This more elegant code comes with a “for experts only” warning.)

```
# Add your code here.
```

For the moment I want to focus on the less elegant (“uglier”) code because it highlights better the key elements of a ggplot2 plot:

1. The first input is the data (stored in a data frame).
2. The second input is an “aesthetic mapping”, created using `aes` that defines how columns are mapped to features of the plot (axes, shapes, colors, *etc*).
3. A “geom”, short for “geometric object”, specifies the type of plot. ggplot2 has an excellent on-line reference at ggplot2.tidyverse.org explaining all the “geoms”, from bar charts to contour plots, with code examples for each.
4. ggplot2 outputs a *ggplot object* `p`, which can be drawn to the screen with `print(p)` or just `p` (then hit “enter” or “return”).

The distinguishing feature of ggplot2 is that plots are created by *adding layers*. This layering allows for infinite variety of plots to be created. The layering approach means that ggplot2 is easily extendible, and many R packages have been developed to enhance ggplot2. (We will use two of these packages, *ggrepel* and *cowplot*.)

A few improvements

No plot is ever right the first time. *What are ways we can improve our plot?* Add code for your improved plot here:

```
# Add your code here.
```

In ggplot, plots can be improved either by modifying the inputs to the functions you are already using (for example, `geom_point` has many settings that can be fiddled with), or by *adding layers*. The ggplot online reference has many helpful examples that illustrate the variety of ways plots can be improved.

Save your work

We’ve made some progress. This is a good point to save our work in an image file that can be shared with others. *What type of image file should we use?*

```
# Add your code here.
```



D. melanogaster

Plot the best-fit line

It has been estimated that an increase of 28 lbs in a dog's body weight corresponds to about a 1-year drop in expected lifespan (with a maximum lifespan of about 13 years). How well does this estimate agree with our data? Let's investigate this question by plotting the line that *best fits* these data (specifically, this is the "least-squares" fit).

```
# Add your code here.
```

Now let's add this "best fit" line to the plot. In case we make a mistake here, let's call our new plot "p2" to avoid writing over our previous plot:

```
# Add your code here.
```

Now let's add another another "abline" layer to compare our estimate against the previous estimate (and name the new plot object "p3"):

```
# Add your code here.
```

Notice how easy it was to add layers to an existing plot!

Which breeds fit the trend, and which don't?

It would be helpful if we could tell which breeds are being plotted. Adding text labels to a ggplot is also done by adding a layer.

There's one catch here—there simply isn't enough real estate on the plot to accommodate all the breed names. This is a great opportunity to play around with a clever package, **ggrepel**, that adds the labels in a way that makes them more readable, and only adds them to the plot when possible. Let's appreciate how simply this sophisticated plot is created.

```
# Add your code here.
```

Notice that the labels are automatically redrawn as the plot is resized. Give it a try!

A QTL for weight

In the *Genetics* paper, the strongest *quantitative trait locus* (QTL) for weight was a QTL on chromosome 7. Using your ggplot coding skills, create a scatterplot to visualize the relationship between weight and the allele frequency at this QTL.

```
# Add your code here.
```



O. bimaculoides

Exploration: a surprising subtlety with color

So far, we have focussed on using the co-ordinate plane to visualize relationships. But other “aesthetics”—color, size, shape, *etc*—can also be used to tell an evocative story. In the last chapter of this tutorial, we will explore the use of color to visualize relationships. In so doing, we will discover a complication.

We learned that the “shortcoat” column had values of 0 or 1 (with a few NAs).

Let’s try varying the color of the points using the shortcoat column:

```
p <- ggplot(dogs, aes(x = weight, y = aod, color = shortcoat)) +  
  geom_point() +  
  theme_cowplot()  
print(p)
```

Are we happy with this plot? How could it be improved? Write your code for an improved plot here:

```
# Add your code here.
```

Optional: Try varying the shape of the points instead of color. If you have have written your ggplot code well, this should only involve a slight change to your code. You can use `scale_shape_manual` to select the shapes. Running `plot(0:23, pch = 0:23)` will give you the full list of shapes to choose from.

```
# Add your code here.
```



More on color

Carefully chosen colors can often be the difference between an effective visualization and an ineffective one. One of the few complaints I have with ggplot2 is that the default colors choices are quite bad. (And, perhaps unsurprisingly, these bad defaults frequently appear in published papers!)

What are the ggplot2 defaults? Let’s check (here I’m reusing some example code from the ggplot2 reference website that uses the “diamonds” data set):

```
data(diamonds)  
n <- nrow(diamonds)  
diamonds <- diamonds[sample(n, 1000),]  
p <- ggplot(diamonds, aes(x = carat, y = price, color = clarity)) +  
  geom_point() +  
  theme_cowplot()  
print(p)
```

If your labmates or co-authors are using these colors, please gently remind them to override the ggplot2 defaults!

Let's now override the ggplot2 default choices using the `scale_color_manual` layer:

```
# Add your code here.
```

The function that controls the color of a discrete variable has an odd name: `scale_color_manual`. This is because, in ggplot2, all methods that control the mapping of variables to colors, shapes, sizes, axes, etc, start with `scale_`.

There are several different ways to specify colors. I prefer specifying colors by name; to get the full list of color names, run `colors()`.

In short, when you use ggplot2, you will often need to make adjustments to the colors. One rule-of-thumb in color choice is that “warmer” colors such as red and orange tend to draw the reader's attention. Use this rule-of-thumb to highlight the diamonds with the best (IF) clarity:

```
# Add your code here.
```

What are other ways you could highlight the diamonds with the best clarity?

The “clarity” column is an example of an *ordered factor*, and it turns out that ggplot2 treats ordered and unordered factors differently. (Most factors in R are unordered.) What are the default colors for an *unordered* factor?

```
# Add your code here.
```

There are several good resources on use of color in data visualization, and I will mention a couple here: Color Brewer (<https://colorbrewer2.org>); and a short article, “Color blindness”, written by Bang Wong (*Nature Methods*, 2011, doi:10.1038/nmeth.1618). I also recommend the [Fundamentals of data visualization](#) book by Claus Wilke.

How would you apply Color Brewer's recommendations to plot ordered or unordered factors? Write some code here to use one of Color Brewer recommended palettes:

```
# Add your code here.
```

Also explore the different options on the Color Brewer website, and see what happens when you restrict to “colorblind safe” palettes.

Obviously, there is so much more to ggplot2. But once you are comfortable with these basic elements, you will find that almost everything else in ggplot2 is a variation of what we covered in this lesson.



T. thermophila

Programming challenge: Joining the Frictionless Research Exchange

I've created a new Data Visualization Programming Challenge this year: it is less structured than in past years, but I hope it will be more fun and stimulate discussion amongst your teammates.

Before the workshop, I've asked each of you to identify and download the *source data* for a figure in a research paper, ideally in an area of research you are interested in, but it doesn't have to be. (It doesn't even have to be a biology paper.) If you are having trouble finding the source data for a published figure, I recommend searching for *eLife* papers ([eLife](#) encourages sharing the source data) and [Zenodo](#), where many authors share the data used to generate the results for a paper.

The programming challenge is a team exercise (each team will submit a single set of solutions), but at the outset I want each of you to find a data set. Then you can discuss amongst your teammates what are the most promising or most interesting data sets to focus on for the programming challenge.

There are three parts to the programming challenge.

In the first part, the goal is to import the data into R, and use `ggplot2` (as well as potentially `ggplot2` extensions such as `ggrepel`) to create a plot that reproduces as closely as possible the plot in the paper. This may involve some manipulation of the data to get it right.

In the second part, you will modify the figure to make it more accessible to people that are color blind. This could involve using “colorblind safe” colors, or by replacing the color with another visualization element. (If the original figure does not use any color, then this second part can be skipped.)

In the third part, you will visualize the data in a different way with the goal of either improving the visualization, or highlighting a different result from the same source data. The new visualization could involve: (i) using a different type of plot (e.g., bar chart, box plot, violin plot, heatmap); (ii) using different plotting elements (e.g., color, shape, size, facets); and/or (iii) using different parts of the source data from what was used in the original plot. There is a lot of freedom in this third part—the only requirement is that you make use of the source data file that was downloaded.

To submit your solutions, you will submit not only the figures you have created, *but also a script that loads the data and recreates the plots*. (The script should also describe in a comment where the data were downloaded from.) The script should save the plots as *files* (e.g., PDF or PNG files). In so doing, you have participated in the *Frictionless Research Exchange* (FRX) and have engaged in practices that keeps the FRX going. (My hope is that you will continue with these practices in your own research!) Points will be awarded for (i) creating plots that meet these goals, (ii) code that successfully reproduces these plots, and (iii) creativity.

You will need to complete these steps for two different published figures (and accompanying source data). You can decide amongst your teammates how best to proceed, but you may want to split the team into two groups, and each group can focus on one data set. One group can act as “peer reviewers” for the other group; they can comment on and suggest improvements to the plots, and verify that that scripts successfully reproduce the plots.

To illustrate what this could look like, in the GitHub repository I've provided an R script, `marino_fig4.R`, that reanalyzes data from a [paper](#) published in *eLife* that studied the relationship between effective population size and genome size. (The GitHub repository also has the source data.)